

claude-windows / f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb\subagents\workflows\wf_35b735a9-0a9\agent-ad900d31668a798c8.jsonl`
- SHA-256: `7ab94d5cb35d7bd6ee41aef32f56f9cf0a10731b73b381a28c2d003d65d9f71b`
- Source modified: `2026-06-21T15:02:09+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_35b735a9-0a9`
- session_id: `f7103c4d-c406-4c6f-9a8a-f4d47f2e54eb`
```

Transcript

1. user (2026-06-21T14:59:20.108Z)

Review the UNCOMMITTED change on branch serving/exposure-safety-preflight in C:\Users\avidu\Projects\badminton-highlight-indexer.

WHAT CHANGED: backend/config/startup.py gained an exposure-safety posture. A new `_exposure_checks(config)` returns findings gated on `security.edge_auth_enabled` (default `False => []` strict no-op). When ON: `EXPOSED_NO_VIDEO_ROOT` (ERROR if `security.allowed_video_root` unset/blank), `EXPOSED_EDGE_AUTH_INCOMPLETE` (ERROR if `cf_team_domain` or `cf_access_aud` missing), `EXPOSED_AUTH_EXTRA_MISSING` (WARN if PyJWT not importable via `_auth_extra_available()`).

`run_startup_checks` gained `include_exposure` (default `False`); `enforce_startup_checks` threads it; backend/main.py `_enforce_startup_or_exit` passes `include_exposure=True` (server only); the worker (backend/worker/__main__.py `_run_startup_checks`) does NOT pass it. Tests in tests/test_startup_checks.py. Tracker/changelog in docs/COMPUTE_DECOUPLED_SERVING/README.md. Read the actual files (git diff is uncommitted in the working tree). For supporting facts, also read backend/api/auth.py (`resolve_tenant`), backend/main.py (the `/api/process` `allowed_video_root` enforcement ~line 294-311, the static/highlights/compile routes), backend/config/models.py `SecurityConfig` (~line 280-306).

Run 'git diff' in C:\Users\avidu\Projects\badminton-highlight-indexer to see the exact change. Be adversarial: prefer finding a real hole over rubber-stamping. Default to a finding if uncertain.

LENS = SECURITY / BYPASS. Try hard to defeat the purpose of this gate. Specifically investigate and rule in/out:

1. Is `edge_auth_enabled` the RIGHT exposure signal? If the owner exposes ONLY via the external CF-Access gate but leaves `edge_auth_enabled=False` (no in-app JWT verify), this check NEVER fires ? is that a real coverage gap for the tunnel-to-a-box topology, and should the doc/runbook call it out? (Don't demand a new config field unless truly warranted.)
2. Does `allowed_video_root` being merely `*set*` give a FALSE sense of safety? e.g. set to `'/'` or `'C:\'` or a root that contains the whole disk ? the gate passes but it's not really a jail. Read backend/main.py's `validate_input_path` / `resolve_under_root` to see what containment it actually enforces.
3. Are there OTHER tenant-reachable file vectors the jail does NOT cover when exposed? `/api/compile`, `/api/highlights/{video_id}/{filename}` (path traversal?), the chunked `upload_dir`, static file serving. Does requiring `allowed_video_root` actually protect all of them, or only `/api/process`? If only `/api/process`, is the `EXPOSED_NO_VIDEO_ROOT` message overclaiming?
4. Could the boot check be a false-NEGATIVE: pass at boot, but the engine still serves unsafely (e.g. `allowed_video_root` set but `upload_dir` outside it, per the `SecurityConfig` comment about `upload_dir`)?

Return concrete findings with file:line.

2. assistant (2026-06-21T14:59:24.151Z)

I'll conduct a systematic security review of this exposure-safety-preflight change. Let me start by examining the uncommitted diff and then investigating each of your concern areas.

3. user (2026-06-21T14:59:24.816Z)

```

warning: in the working copy of 'backend/config/startup.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'tests/test_startup_checks.py', LF will be replaced by CRLF the
next time Git touches it
diff --git a/backend/config/startup.py b/backend/config/startup.py
index b16f0de..28a5858 100644
--- a/backend/config/startup.py
+++ b/backend/config/startup.py
@@ -1,4 +1,4 @@
-"""Per-profile startup checks (COMPUTE_DECOUPLED_SERVING M5).
+"""Startup checks ? per-profile coherence (M5) + exposure-safety posture (M9 hardening).

A deployment profile (`truly-local` / `cloud-serving` / `cpu-mock`) implies a coherent
runtime. These checks fail/warn fast at process startup ? a loud signal before any
@@ -15,10 +15,20 @@ runtime. These checks fail/warn fast at process startup ? a loud signal
*b
    a warning: Application Default Credentials can come from the metadata server / gcloud,
which
    we can't probe offline, so the storage backend's own init is the authority; we just flag
it.

-DEFAULT-OFF: with no profile selected (`profile == "") this is a strict no-op ?
exactly
-the pre-M5 behaviour. Only an explicitly-selected profile triggers any check (mirrors the
loader,
-which applies a preset only for an explicit profile). So nothing changes for today's manual-
knob
-deployments.
+Exposure-safety posture (server only, `include_exposure=True`). When the engine is
exposed
+behind an edge gate (the owner flips `security.edge_auth_enabled` on for the Cloudflare-
Access
+boundary ? the alpha-shareable "tunnel-to-a-box" topology), the gate is only half the safety
story.
+A half-configured exposure starts looking healthy then leaks / 500s per request, so we verify
the
+rest of the posture at boot: the `allowed_video_root` path-jail is set (else an
authenticated
+tenant could read an arbitrary local file ? error), the CF team-domain + AUD are present
(else
+no token can verify ? error, lifting the current first-request failure to boot), and the
+`auth` extra is importable (else every request 500s ? warning). Only the HTTP server
passes
+`include_exposure=True`; the worker reads `--video-uri` (not untrusted tenant paths), so it
is
+never "exposed" and stays unaffected.
+
+DEFAULT-OFF (both dimensions): with no profile selected (`profile == "") the profile
checks are
+a strict no-op ? exactly the pre-M5 behaviour; with `edge_auth_enabled=False` (today's
default)
+the exposure checks are a strict no-op too. So nothing changes for today's manual-knob
deployments.
"""

from __future__ import annotations
@@ -66,30 +76,100 @@ def _creds_discoverable(env: Mapping[str, str]) -> bool:
    return any(env.get(k) for k in _GCP_ENV_HINTS)

+def _auth_extra_available() -> bool:
+    """True if PyJWT (the `auth` extra) is importable ? probed WITHOUT importing it
(find_spec),
+    so this stays cheap + side-effect-free. Module-level so tests can monkeypatch it
deterministically

```

```

+ (whether PyJWT is installed in the test env must not change the WARN's presence)."""
+ try:
+     import importlib.util
+     return importlib.util.find_spec("jwt") is not None
+ except Exception: # pragma: no cover - find_spec only raises on a broken parent package
+     return False
+
+
+def _exposure_checks(config: Any) -> list[StartupCheck]:
+    """Exposure-safety posture for an *exposed* HTTP server. Gated on
+    ``security.edge_auth_enabled``
+    (default False ? ``[]`` = strict no-op, today's behaviour). When edge auth is ON ? the "I
+    am
+    exposing this engine behind the CF-Access gate" signal ? the gate alone isn't enough:
+    verify the
+    rest of the posture at boot so a half-configured exposure fails fast instead of leaking /
+    500-ing
+    per request. Config-only (no IO beyond the ``find_spec`` auth-extra probe)."""
+    sec = getattr(config, "security", None)
+    if not sec or not getattr(sec, "edge_auth_enabled", False):
+        return [] # DEFAULT-OFF: edge auth off ? no posture enforcement (byte-identical to
+        today).
+
+    out: list[StartupCheck] = []
+
+    # (1) The path-jail is load-bearing once authenticated tenants can hit /api/process:
+    WITHOUT it,
+    # a tenant can point the engine at an ARBITRARY local file (allowed_video_root unset = "any
+    local
+    # file", the single-user default). Fatal for an exposed engine.
+    root = getattr(sec, "allowed_video_root", None)
+    if not root or not str(root).strip():
+        out.append(StartupCheck(
+            LEVEL_ERROR, "EXPOSED_NO_VIDEO_ROOT",
+            "security.edge_auth_enabled is ON (engine exposed) but security.allowed_video_root
+            is "
+            "unset ? an authenticated tenant could make /api/process read an arbitrary local
+            file. "
+            "Set allowed_video_root to a jail directory before exposing the engine.",
+        ))
+
+    # (2) Edge auth cannot verify a token without the team domain + AUD. Today that only fails
+    on the
+    # FIRST request (auth.resolve_tenant); lift it to BOOT so a misconfigured gate never starts
+    # "healthy" then 401s every request.
+    team = (getattr(sec, "cf_team_domain", "") or "").strip()
+    aud = (getattr(sec, "cf_access_aud", "") or "").strip()
+    if not team or not aud:
+        missing = " + ".join(n for n, v in (("cf_team_domain", team), ("cf_access_aud", aud))
+        if not v)
+        out.append(StartupCheck(
+            LEVEL_ERROR, "EXPOSED_EDGE_AUTH_INCOMPLETE",
+            f"security.edge_auth_enabled is ON but {missing} is not configured ? the Cf-Access
+            JWT "
+            "cannot be verified (every request would fail). Set the CF team domain + Access
+            AUD.",
+        ))
+
+    # (3) The auth extra (PyJWT[crypto]) is lazy-imported by auth.verify_cf_access_token. If
+    it's
+    # missing, every request 500s at verify time ? a boot WARN beats a per-request 500.
+    Warning, not
+    # fatal: the auth module's own ImportError is the final authority.
+    if not _auth_extra_available():
+        out.append(StartupCheck(

```

```

+         LEVEL_WARN, "EXPOSED_AUTH_EXTRA_MISSING",
+         "security.edge_auth_enabled is ON but PyJWT[crypto] is not importable ? Cf-Access "
+         "verification will fail at request time. Install the 'auth' extra (pip install -e
+         .[auth]).",
+         ))
+
+     return out
+
+
+ def run_startup_checks(
+     config: Any,
+     *,
+     cuda_available: Optional[bool] = None,
+     env: Optional[Mapping[str, str]] = None,
+     include_exposure: bool = False,
+ ) -> list[StartupCheck]:
+     """Return the per-profile findings for ``config`` (empty when no profile is selected).
+     """Return the startup findings for ``config``.

+     ``cuda_available`` is the caller's best-effort GPU probe (``None`` = not probed ? GPU
checks
+     are skipped; the API server passes ``None``, the worker passes
+     ``presets.probe_cuda_available()``).
+     - Pure: no IO, no torch import ? the caller owns the probe so this stays importable
everywhere.
+     ``include_exposure`` adds the exposure-safety posture (``_exposure_checks``) ? the HTTP
**server**
+     passes ``True``; the worker leaves it ``False`` (it is never "exposed"). Both dimensions
are
+     DEFAULT-OFF: no profile ? no profile checks; ``edge_auth_enabled=False`` ? no exposure
checks.
+     Pure: no IO (beyond the ``find_spec`` auth probe), no torch import ? the caller owns the
GPU probe.
+     """
+     e = os.environ if env is None else env
+     checks: list[StartupCheck] = []
+
+     # Exposure-safety posture ? INDEPENDENT of deployment.profile (a tunnel-to-a-box exposure
often
+     # runs with no profile set), gated on edge_auth_enabled (default-OFF). Server-only.
+     if include_exposure:
+         checks.extend(_exposure_checks(config))
+
+     profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
+     if not profile:
+         - return [] # DEFAULT-OFF: no profile ? no checks (identical to pre-M5).
+         return checks # no profile ? only the exposure posture (itself off unless edge auth is
on).

+
+     e = os.environ if env is None else env
+     storage_backend = getattr(getattr(config, "storage", None), "backend", "local")
+     indexing = getattr(config, "indexing", None)
+     detector_impl = getattr(indexing, "detector_impl", "auto")
+     device = getattr(getattr(indexing, "wasb", None), "device", "cuda")

+     checks: list[StartupCheck] = []
+
+     if profile == "truly-local":
+         # truly-local reads bytes from the local FS or a *mounted* Drive ? a cloud backend here
is
+         # an inconsistent (but not fatal) choice worth surfacing.
@@ -162,15 +242,18 @@ def enforce_startup_checks(
+     cuda_available: Optional[bool] = None,
+     env: Optional[Mapping[str, str]] = None,
+     raise_on_error: bool = True,

```

```

+ include_exposure: bool = False,
+ logger: Optional[logging.Logger] = None,
) -> list[StartupCheck]:
    """Run the checks, LOG each (errors at ERROR, warnings at WARNING), and ? unless
    ``raise_on_error=False`` ? raise ``StartupCheckError`` if any FATAL check tripped.

- Returns the full findings list (so a caller can inspect/surface them). A no-op with no
logging
- and no raise when no profile is selected (``run_startup_checks`` returns ``[]``)."""
+ ``include_exposure`` (server-only) adds the exposure-safety posture. Returns the full
findings
+ list (so a caller can inspect/surface them). A no-op with no logging and no raise when both
+ dimensions are off (``run_startup_checks`` returns ``[]``)."""
    log = logger or logging.getLogger(__name__)
- checks = run_startup_checks(config, cuda_available=cuda_available, env=env)
+ checks = run_startup_checks(
+     config, cuda_available=cuda_available, env=env, include_exposure=include_exposure)
    for c in checks:
        if c.level == LEVEL_ERROR:
            log.error("[STARTUP] %s: %s", c.code, c.message)
diff --git a/backend/main.py b/backend/main.py
index 6768d30..0aec8e3 100644
--- a/backend/main.py
+++ b/backend/main.py
@@ -562,13 +562,15 @@ def run_cli_diagnose_clip(video_path: str, timestamp: float):

    def _enforce_startup_or_exit(config) -> None:
-        """M5: run the per-profile startup checks for the server/CLI path and ``sys.exit(1)`` on a
-        FATAL incoherent profile (e.g. cloud-serving pointed at local-only storage) BEFORE any
work.
-        A strict no-op when no ``deployment.profile`` is selected (today's behaviour). No GPU probe
-        here ? the server stays torch-light; the detector healthcheck is the GPU authority.
Extracted
-        so the fail-fast wiring + the default-OFF no-op are unit-testable."""
+        """M5: run the startup checks for the server path and ``sys.exit(1)`` on a FATAL finding
BEFORE
+        any work ? a fatal incoherent profile (e.g. cloud-serving pointed at local-only storage)
OR, for
+        the exposed server (``include_exposure=True``, M9 hardening), a half-configured exposure
(edge
+        auth on without the ``allowed_video_root`` jail / CF team+AUD). A strict no-op when no
+        ``deployment.profile`` is selected AND edge auth is off (today's behaviour). No GPU probe
here ?
+        the server stays torch-light; the detector healthcheck is the GPU authority. Extracted so
the
+        fail-fast wiring + the default-OFF no-op are unit-testable."""
        try:
-            enforce_startup_checks(config, logger=logger)
+            enforce_startup_checks(config, logger=logger, include_exposure=True)
        except StartupCheckError as e:
            logger.error("[STARTUP] refusing to start: %s", e)
            sys.exit(1)
diff --git a/docs/COMPUTE_DECOUPLED_SERVING/README.md b/docs/COMPUTE_DECOUPLED_SERVING/README.md
index 3b3b1c5..f59f5d9 100644
--- a/docs/COMPUTE_DECOUPLED_SERVING/README.md
+++ b/docs/COMPUTE_DECOUPLED_SERVING/README.md
@@ -121,7 +121,7 @@ Legend: ? Todo · ? In progress · ? Done · ? Gated. **One small PR per
| M6 | Cloud Run GPU **dispatch shim** (control plane ? job via storage ? re-claim) +
**containerized worker image** (ffmpeg+CUDA+WASB+fonts); mocked-dispatch tests | M5 | ? owner
(GCP spend) | ? | [#287](https://github.com/Khelsutra/badminton-highlight-indexer/pull/287)
(shim+image+mocked ?; real build/run = M7 ? owner) |
| M7 | **cloud-serving profile** e2e on L4: upload<2GB + gdrive/bucket ? detect+stitch on
Cloud Run ? reel via signed URL; **DEPLOY_REPORT** (posterity, sample runs + contact sheet) | M6
| ? owner (spend) | ? | ? |

```

| M8 | Per-job **cost ledger** (v6 migration: `owner\_id, gpu\_active\_seconds, wall\_seconds, bytes\_out, cost\_usd, cost\_breakdown\_json`) + **pricebook** + aggregation + `/api/.../cost` | M7 | ? owner (billing) | ? | [#288](https://github.com/Khelsutra/badminton-highlight-indexer/pull/288) (ledger+pricebook+cost ? default-OFF; real prices + usage-emission ? owner/follow-up) |

-| M9 | **Edge auth** (Cf-Access extraction) + per-tenant scoping on `user\_id`; DPA/consent gate wiring | M7 | ? owner (privacy/counsel) | ? | [#290](https://github.com/Khelsutra/badminton-highlight-indexer/pull/290) (auth: Cf-Access JWT verify + owner_id tag, default-OFF ?; consent cols + ToS/DPA ? owner/counsel) |

+| M9 | **Edge auth** (Cf-Access extraction) + per-tenant scoping on `user\_id`; DPA/consent gate wiring | M7 | ? owner (privacy/counsel) | ? | [#290](https://github.com/Khelsutra/badminton-highlight-indexer/pull/290) (auth: Cf-Access JWT verify + owner_id tag, default-OFF ?; **exposure-safety boot posture** in `startup.py` ? edge-auth-on requires the `allowed\_video\_root` jail + CF team/AUD or the server refuses to start, see changelog; consent cols + ToS/DPA ? owner/counsel) |

| M10 | `byo\_worker` / zero-infra-BYO ? **design-only, DEFERRED** | M8,M9 | ? explicit owner approval | ? | ? |

| **M11** | **Data-flywheel harness** (D12): **capture** the upload + Gemini reel/rally output ? **reliably store** under the per-video workspace ? **shadow-eval** owned-vs-Gemini (dual-eval-set / `experiment.compare`) ? **promote** good ones to the golden TRAINING set (human-reviewed labels via the annotation flow). Closes the loop so the owned model climbs to the D11 floor. Reuses `data\_pipeline/` + `backend/eval/` + the consent gate (M9). | M7,M9 | ? owner (consent/privacy) | ? | ? |

| **M12** | **gdrive-api** (OAuth) StorageBackend (D14 Northstar): **cloud reads the source from + writes the stitched reel back to the user's Google Drive next to the source** (per-user OAuth consent) ? the mobile/cloud-native Drive round-trip. Distinct from the local mount backend; slots into the `StorageBackend` ABC. | M7,M9 | ? owner (OAuth/consent) | ? | ? |

@@ -167,3 +167,4 @@ Legend: ? Todo · ? In progress · ? Done · ? Gated. **One small PR per**

- 2026-06-21 ? **M8 code landed** ([#288](https://github.com/Khelsutra/badminton-highlight-indexer/pull/288)): per-job **cost ledger** ? **backend/billing.py** (pure `usage × pricebook` math, USD internal / INR display) + a **v6 migration** (NULLABLE cost columns on `jobs` + a versioned `pricebook` table seeded `placeholder-v0` at \$0; additive-NONE, idempotent) + **record_job_cost** / **get_job_cost** / **get_video_cost** + **GET /api/jobs/{id}/cost** + **/api/videos/{id}/cost** + **BillingConfig** (**default-OFF**) + a gated **maybe_record_job_cost** hook. Adversarial review (`wf\_498b24af`) folded a **major silent-GPU-under-bill** (an unpriced **gpu_type** is now surfaced, not invisible \$0) + the **honest no-op** caveat (usage emission is a follow-up ? cost 0 until wired) + breakdown reconciliation. Suite green, ruff+mypy clean. **M8** ? ? owner inserts real GCP prices (a new pricebook version) + the worker usage-emission is a follow-up; placeholder ships \$0 = no user-facing change. v6 is the shared bump (M9-consent cols extend it).
- 2026-06-21 ? **M6 code landed** ([#287](https://github.com/Khelsutra/badminton-highlight-indexer/pull/287)): **backend/compute** / **ComputeBackend** registry ? **CloudRunCompute** dispatches a Cloud Run **Job** (injectable client) then **bucket-polls** a done-marker via **execution_policy.poll_until** + the storage layer (D16); **get_compute_backend** returns a backend ONLY for **compute_target=cloud_run** (else **None** ? the worker's in-process path, **default-OFF**). The dispatched container runs **inline** (forced **local_gpu**+native, never re-dispatch); the worker writes a **done-uri** completion marker on success. **deploy/cloudrun/{Dockerfile,README}** = the L4 image (CUDA+ffmpeg+fonts+wasb` conda env; weights staged per WASB-C3) + the M7 runbook. Adversarial review (`wf_87b22fa4`) folded a **blocker** (cloud `fetch` needs a dst ? reads silently returned `{}` on gcs/s3) + a poll-wait test gap + **PolicyTimeout**?clean rc 4. Suite green, ruff+mypy clean. **M6** ? ? the real build/push/L4 run + **DEPLOY_REPORT** is M7 (owner, D17 budget).
- 2026-06-21 ? **M5 code landed** ([#286](https://github.com/Khelsutra/badminton-highlight-indexer/pull/286)): **backend/config/startup.py** ? per-profile fail/warn-fast startup checks (cloud-serving + non-cloud storage = the one fatal coherence error; GPU/creds mismatches = warnings, the runner healthcheck stays the authority), wired into the server (**enforce_startup_or_exit** ? exit 1) + the worker (**cmd_infer** / **cmd_train** ? rc 2), default-OFF (probe gated behind an active profile ? the worker stays torch-free at **profile=""**). **L4 profile-parity test** (**tests/test_profile_parity.py**): same stub trajectory ? byte-identical windows + segment ranges across truly-local / cpu-mock / no-profile + worker-vs-in-process. Adversarial 4-lens review folded (`wf\_34671ac1`). Suite green, ruff+mypy clean. **M5** ? ? the real-GPU truly-local runlog (L5) is owner-side (template pre-staged in **runlogs**).
- + 2026-06-21 ? **M9 hardening: exposure-safety boot posture** (alpha-shareable "tunnel-to-a-box" enabler): extended **backend/config/startup.py** with a **server-only exposure posture** (new **include_exposure=True** from **enforce_startup_or_exit**; the worker stays unaffected ? it reads

`--video-uri`, not untrusted tenant paths). Gated on `security.edge\_auth\_enabled` (**default-OFF ? strict no-op**, byte-identical to today): when the owner flips edge auth ON to expose the engine behind the CF-Access gate, three checks fire at **boot** ? `EXPOSED\_NO\_VIDEO\_ROOT` (**fatal**: `allowed\_video\_root` jail unset ? a tenant could read an arbitrary local file), `EXPOSED\_EDGE\_AUTH\_INCOMPLETE` (**fatal**: CF team-domain/AUD missing ? lifts the current first-request 401 to boot), `EXPOSED\_AUTH\_EXTRA\_MISSING` (**warn**: `PyJWT[crypto]` not importable ? would 500 per request). So a half-configured exposure fails fast instead of leaking/500-ing. +13 unit tests (default-OFF no-op, worker-ignores, both-fatals-compose-with-profile-checks, server-exit). Pairs with the appliance **T3** network posture (raw `:8000` refused ? owner-run). Suite green, ruff+mypy clean. **? backstops M9 ? the actual exposure flip + CF team/AUD/ingress + key rotation stay owner-side.**

```
diff --git a/tests/test_startup_checks.py b/tests/test_startup_checks.py
index be250a5..95b9eba 100644
```

```
--- a/tests/test_startup_checks.py
```

```
+++ b/tests/test_startup_checks.py
```

```
@@ -9,6 +9,7 @@ import logging
```

```
import pytest
```

```
from backend.config import AppConfig
```

```
+from backend.config import startup as startup_mod
```

```
from backend.config.startup import (
```

```
    LEVEL_ERROR,
```

```
    LEVEL_WARN,
```

```
@@ -29,6 +30,19 @@ def _cfg(profile="", backend="local", device="cuda", detector_impl="auto"):
```

```
    })
```

```
+def _exposed_cfg(*, edge_auth=True, video_root="/jail",
```

```
team="https://acme.cloudflareaccess.com",
```

```
+    aud="abc123", profile="", backend="local"):
```

```
+    """An AppConfig with the security/exposure fields set, for the exposure-posture checks."""
```

```
+    sec = {"edge_auth_enabled": edge_auth, "cf_team_domain": team, "cf_access_aud": aud}
```

```
+    if video_root is not None:
```

```
+        sec["allowed_video_root"] = video_root
```

```
+    return AppConfig.model_validate({
```

```
+        "deployment": {"profile": profile},
```

```
+        "storage": {"backend": backend},
```

```
+        "security": sec,
```

```
+    })
```

```
+
```

```
+
```

```
def _codes(checks):
```

```
    return {c.code for c in checks}
```

```
@@ -224,3 +238,110 @@ def test_server_enforce_does_not_exit_on_warnings():
```

```
    """A warning-only profile (truly-local + cloud storage) must NOT abort the server."""
```

```
    import backend.main as bmain
```

```
    assert bmain._enforce_startup_or_exit(_cfg(profile="truly-local", backend="gcs")) is None
```

```
+
```

```
+
```

```
+# ----- exposure-safety posture (M9 hardening, server-only)
```

```
+
```

```
+def test_exposure_off_by_default_is_noop():
```

```
+    """edge_auth_enabled=False (today's default) ? ZERO exposure checks, even with
```

```
include_exposure
```

```
+    and an otherwise-unsafe config (no allowed_video_root). The strict default-OFF guarantee."""
```

```
+    cfg = _exposed_cfg(edge_auth=False, video_root=None, team="", aud="")
```

```
+    assert run_startup_checks(cfg, include_exposure=True, env={}) == []
```

```
+
```

```
+
```

```
+def test_exposure_not_run_without_flag(monkeypatch):
```

```
+    """The worker path (include_exposure=False, the default) NEVER runs exposure checks ? even with
```

```

+     edge auth ON and an unsafe config. Only the HTTP server opts in."""
+     monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
+     cfg = _exposed_cfg(edge_auth=True, video_root=None, team="", aud="")
+     assert run_startup_checks(cfg, env={}) == [] # include_exposure defaults False
+
+
+def test_exposed_without_video_root_is_fatal(monkeypatch):
+     monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
+     cfg = _exposed_cfg(video_root=None) # team+aud set, jail missing
+     checks = run_startup_checks(cfg, include_exposure=True, env={})
+     assert _codes(checks) == {"EXPOSED_NO_VIDEO_ROOT"}
+     assert checks[0].level == LEVEL_ERROR
+
+
+@pytest.mark.parametrize("video_root", [None, "", " "])
+def test_exposed_blank_video_root_is_fatal(monkeypatch, video_root):
+     """None, empty, and whitespace-only roots are all treated as 'unset' ? fatal."""
+     monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
+     cfg = _exposed_cfg(video_root=video_root)
+     assert "EXPOSED_NO_VIDEO_ROOT" in _codes(run_startup_checks(cfg, include_exposure=True,
+env={}))
+
+
+@pytest.mark.parametrize("team,aud,expect_missing", [
+     ("", "abc", "cf_team_domain"),
+     ("https://t.cloudflareaccess.com", "", "cf_access_aud"),
+     ("", "", "cf_team_domain + cf_access_aud"),
+])
+def test_exposed_incomplete_cf_config_is_fatal(monkeypatch, team, aud, expect_missing):
+     monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
+     cfg = _exposed_cfg(team=team, aud=aud) # jail set, CF config partial
+     checks = run_startup_checks(cfg, include_exposure=True, env={})
+     assert _codes(checks) == {"EXPOSED_EDGE_AUTH_INCOMPLETE"}
+     assert checks[0].level == LEVEL_ERROR
+     assert expect_missing in checks[0].message # names exactly what's missing
+
+
+def test_exposed_missing_auth_extra_warns(monkeypatch):
+     """PyJWT not importable ? a WARN (never fatal ? the auth module's ImportError is the
+authority)."""
+     monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: False)
+     cfg = _exposed_cfg() # fully configured otherwise
+     checks = run_startup_checks(cfg, include_exposure=True, env={})
+     assert _codes(checks) == {"EXPOSED_AUTH_EXTRA_MISSING"}
+     assert checks[0].level == LEVEL_WARN
+
+
+def test_exposed_fully_configured_is_clean(monkeypatch):
+     monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
+     cfg = _exposed_cfg() # edge auth on, jail set, team+aud set, auth extra present
+     assert run_startup_checks(cfg, include_exposure=True, env={}) == []
+
+
+def test_exposed_both_errors_together(monkeypatch):
+     """An unsafe exposure trips BOTH fatal checks at once (jail missing + CF config
+missing)."""
+     monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
+     cfg = _exposed_cfg(video_root=None, team="", aud="")
+     codes = _codes(run_startup_checks(cfg, include_exposure=True, env={}))
+     assert codes == {"EXPOSED_NO_VIDEO_ROOT", "EXPOSED_EDGE_AUTH_INCOMPLETE"}
+
+
+def test_exposure_composes_with_profile_checks(monkeypatch):
+     """Exposure posture is INDEPENDENT of deployment.profile ? an exposed cloud-serving config
+with

```

```

+ local storage trips both the exposure jail error AND the profile STORAGE_INCOHERENT
error. """
+ monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
+ cfg = _exposed_cfg(video_root=None, profile="cloud-serving", backend="local")
+ codes = _codes(run_startup_checks(cfg, cuda_available=True, include_exposure=True,
+                               env={"K_SERVICE": "x"}))
+ assert codes == {"EXPOSED_NO_VIDEO_ROOT", "STORAGE_INCOHERENT"}
+
+
+def test_server_enforce_exits_on_unsafe_exposure(monkeypatch):
+    """The server's fail-fast wiring aborts (exit 1) when edge auth is on but the engine is
    exposed
+    without the allowed_video_root jail ? BEFORE serving a single request. """
+    monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
+    import backend.main as bmain
+    with pytest.raises(SystemExit) as ei:
+        bmain._enforce_startup_or_exit(_exposed_cfg(video_root=None))
+    assert ei.value.code == 1
+
+
+def test_server_enforce_allows_safe_exposure(monkeypatch):
+    """A fully-configured exposure starts cleanly (no exit). """
+    monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
+    import backend.main as bmain
+    assert bmain._enforce_startup_or_exit(_exposed_cfg()) is None
+
+
+def test_worker_path_ignores_unsafe_exposure(monkeypatch, tmp_path):
+    """The worker (cmd path) does NOT enforce exposure posture ? an edge-auth-on config that
    would
+    abort the server runs fine on the worker (it reads --video-uri, not untrusted tenant
    paths). """
+    monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
+    from backend.worker import __main__ as wmain
+    cfg_file = tmp_path / "c.json"
+    cfg_file.write_text('{"security": {"edge_auth_enabled": true}}') # exposed-ish, no
    jail/team/aud
+    assert wmain._run_startup_checks(wmain.load_config(str(cfg_file))) is True

```

4. assistant (2026-06-21T14:59:27.027Z)

Now let me examine the key files to investigate the four concern areas you specified.

5. user (2026-06-21T14:59:27.954Z)

```

280
281 @app.post("/api/process", status_code=202)
282 def process_video(req: ProcessRequest, request: Request):
283     """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
284
285     Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
286     client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
287     runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
288     worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
289     """
290     # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local
    single-user,
291     # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a
    missing or
292     # spoofed identity header fails here with 401, before any work) and tags the job's
    owner_id.
293     try:
294         owner_id = edge_auth.resolve_tenant(request.headers, global_config)
295     except edge_auth.EdgeAuthError as e:
296         raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e

```

```

297
298     allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
299     try:
300         validate_input_path(req.video_path, allowed_root=allowed_root)
301         # M2: resolve the input once (path or storage URI) ? raises ValueError for an
unsupported
302         # scheme, which we map to a clean 400 (an unknown scheme must never 500).
303         input_ref = storage.resolve_input_ref(req.video_path, getattr(global_config,
"storage", None))
304     except ValueError as e:
305         raise HTTPException(status_code=400, detail=str(e)) from e
306     # The local-existence fast-fail applies only to a LOCAL input, and stats the
RESOLVED local path
307     # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI
string. A
308     # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified
when the worker
309     # fetches it (a fetch miss fails the job loudly). validate_input_path above still
runs for every
310     # input: its null-byte guard always applies, and a configured allowed_video_root
(hosted mode)
311     # rejects a non-local URI as out-of-root.
312     if input_ref.backend == "local" and not os.path.exists(input_ref.key):
313         raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
314     if req.segmenter and not segmenter_registry.get(req.segmenter):
315         available = list(segmenter_registry.list_available().keys())
316         raise HTTPException(
317             status_code=400,
318             detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}")
319     )
320
321     job_id = uuid.uuid4().hex
322     if not db_instance.create_job(job_id, req.video_path, req.segmenter,
323                                req.player_pool, req.max_frames, owner_id=owner_id):
324         raise HTTPException(status_code=500, detail="Failed to persist job record.")
325     _get_executor().submit(_drain_due_jobs)
326     return JSONResponse(
327         status_code=202,
328         content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
329     )

```

6. user (2026-06-21T14:59:28.881Z)

```

1     """Cloudflare Access edge-auth ? verify the Cf-Access JWT, derive the tenant ``user_id``
(M9, D9).
2
3     **DEFAULT-OFF:** when ``security.edge_auth_enabled`` is False the resolver is a strict
NO-OP (returns
4     ``None`` = the local single-user) ? nothing about the existing local path changes. When
enabled, the
5     ``Cf-Access-Jwt-Assertion`` header is **required** and **verified** (RS256) against the
team JWKS ?
6     signature + exp + issuer + audience ? so a direct hit to the Cloud Run URL **can't
spooft** the
7     identity header (the explicit D9 anti-spoof requirement). Returns the verified
``user_id`` (the
8     token's ``email``, else ``sub``).
9
10    ``jwt`` (``PyJWT[crypto]``) is **lazy-imported** so the base runtime works without it
(install the
11    ``auth`` extra for hosted/multi-tenant). The JWKS fetch is **injectable**, so the whole
verifier is

```

```

12     unit-tested **offline** with an in-test RSA keypair + a fake JWKS ? no network, no
Cloudflare.
13     ""
14
15     from __future__ import annotations
16
17     import json
18     import time
19     import urllib.request
20     from typing import Any, Callable, Mapping, Optional
21
22     # Cloudflare Access sends the signed assertion in this header (it terminates at the CF
edge).
23     CF_ACCESS_HEADER = "Cf-Access-Jwt-Assertion"
24     _JWKS_PATH = "/cdn-cgi/access/certs"
25     _JWKS_TTL_SEC = 600.0 # re-fetch the team JWKS at most every 10 min (keys rotate
slowly)
26
27     JwksFetcher = Callable[[str], dict]
28
29
30     class EdgeAuthError(Exception):
31         """A required Cf-Access token was missing or failed verification ? the caller
returns 401/403."""
32
33
34     # module-level JWKS cache: team_domain -> (fetched_at, jwks_dict). Production only;
tests inject.
35     _jwks_cache: dict[str, tuple[float, dict]] = {}
36
37
38     def _issuer(team_domain: str) -> str:
39         return team_domain.rstrip("/")
40
41
42     def _default_jwks_fetcher(team_domain: str) -> dict:
43         """Fetch the team's public JWKS (`<team_domain>/cdn-cgi/access/certs`). Network ?
only the
44         production path reaches here; tests pass an injected fetcher."""
45         url = _issuer(team_domain) + _JWKS_PATH
46         with urllib.request.urlopen(url, timeout=5) as resp: # noqa: S310 (https team
domain)
47             data: Any = json.load(resp)
48             if not isinstance(data, dict) or "keys" not in data:
49                 raise EdgeAuthError(f"unexpected JWKS shape from {url}")
50             return data
51
52
53     def _get_jwks(team_domain: str, fetcher: Optional[JwksFetcher]) -> dict:
54         """Return the JWKS for `team_domain`, cached with a TTL. An injected `fetcher`
(tests)
55         bypasses the cache so each test is deterministic."""
56         if fetcher is not None:
57             return fetcher(team_domain)
58         now = time.time()
59         cached = _jwks_cache.get(team_domain)
60         if cached and (now - cached[0]) < _JWKS_TTL_SEC:
61             return cached[1]
62         jwks = _default_jwks_fetcher(team_domain)
63         _jwks_cache[team_domain] = (now, jwks)
64         return jwks
65
66
67     def verify_cf_access_token(token: str, *, team_domain: str, aud: str, jwks: dict) ->
str:

```

```

68         """Verify a Cf-Access JWT against ``jwks`` (RS256; checks sig + exp + issuer +
audience) and
69         return the user_id (``email`` else ``sub``). Raises ``EdgeAuthError`` on ANY
failure."""
70         try:
71             import jwt
72             from jwt import PyJWKSet
73         except Exception as exc: # pragma: no cover - only when the auth extra isn't
installed
74             raise EdgeAuthError(
75                 "PyJWT[crypto] is required for edge auth ? install the 'auth' extra "
76                 "(pip install -e .[auth]) on a host with edge_auth_enabled."
77             ) from exc
78
79         try:
80             kid = jwt.get_unverified_header(token).get("kid")
81             if not kid:
82                 raise EdgeAuthError("token header has no kid")
83             signing_key = PyJWKSet.from_dict(jwks)[kid]
84             claims = jwt.decode(
85                 token, signing_key.key, algorithms=["RS256"],
86                 audience=aud, issuer=_issuer(team_domain),
87                 options={"require": ["exp", "iss", "aud"]},
88             )
89         except EdgeAuthError:
90             raise
91         except KeyError as exc: # kid not in the JWKS
92             raise EdgeAuthError(f"no JWKS key for kid {exc}") from exc
93         except Exception as exc: # jwt.* errors: bad sig / expired / wrong aud|iss /
malformed
94             raise EdgeAuthError(f"Cf-Access token verification failed: {exc}") from exc
95
96         user = claims.get("email") or claims.get("sub")
97         if not user:
98             raise EdgeAuthError("verified token carries no email/sub claim")
99         return str(user)
100

```

7. user (2026-06-21T14:59:29.716Z)

```

280     class SecurityConfig(BaseModel):
281         # When set, /api/process video_path must resolve under this directory (hosted/tenant
282         # mode). Default None preserves the single-user local 'any local file' workflow.
283         allowed_video_root: Optional[str] = None
284
285         # --- Chunked upload (B2, PR-2) ---
286         # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
287         # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
288         # contain upload_dir or /api/process will reject the uploaded paths.
289         upload_dir: str = "output/uploads"
290         # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
291         # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
292         max_upload_mb: int = 4096
293         max_part_mb: int = 95
294         allowed_upload_extensions: List[str] = ["mp4", "mov"]
295
296         # --- M9 edge auth (Cloudflare Access, D9) ---
297         # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local
single-user
298         # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-
Jwt-Assertion`
299         # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
300         # header) and tags the job with the verified user_id (owner_id). The team domain
(e.g.

```

```

301         # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is
the CF Access
302         # application's audience tag. NO secrets here ? these are public identifiers.
303         edge_auth_enabled: bool = False
304         cf_team_domain: str = ""          # e.g. https://<team>.cloudflareaccess.com (issuer +
/certs JWKS)
305         cf_access_aud: str = ""          # the CF Access application AUD tag the token must
carry
306         # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at
import in
307         # backend/main.py so importing the app does no filesystem I/O; default [""].
Credentials stay
308         # OFF ? Cf-Access is a header, not a cookie.)
309
310
311     class StorageConfig(BaseModel):
312         """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
313         behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO
Spaces
314         + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a
locally MOUNTED
315         Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to
the constructor ?
316         **secrets are never stored here**, only endpoints / regions / file paths / source-
selectors
317         (boto3/google auth chains supply credentials). See ``backend/storage/``.
318         backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
319         # Output root for produced artifacts. "" => fall back to output.output_dir (non-
breaking).
320         output_root: str = ""
321         # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
322         # Default ``local`` / "" keeps today's behaviour: a bare path / ``local://`` URI is
read IN PLACE
323         # (identity, no copy). Set these to read inputs from a bucket / mounted Drive ? a
bare/relative
324         # input name is resolved against ``input_root`` (e.g.
``input_root="gs://bucket/videos"`` + "x.mp4"
325         # => ``gs://bucket/videos/x.mp4``) or, if ``input_root`` is empty, prefixed with
``input_backend``
326         # (``"gcs"`` + "bucket/x.mp4" => ``gcs://bucket/x.mp4``). An explicit scheme on the
input
327         # (``gs://``/``s3://``) or an absolute local path ALWAYS wins. Non-local inputs
are fetched to a
328         # scratch dir before processing (see ``backend.storage.acquire_local_input``).
329         input_backend: Literal["local", "s3", "gcs", "gdrive"] = "local"

```

8. assistant (2026-06-21T14:59:32.125Z)

Now let me search for the `validate_input_path` and `resolve_under_root` functions to understand the containment logic:

9. user (2026-06-21T14:59:32.519Z)

Found 1 file
backend\utils\paths.py

10. user (2026-06-21T14:59:33.442Z)

```

1     import os
2     import sys
3     import argparse
4     import logging
5     import uuid

```

```

6     from urllib.parse import quote
7     import uvicorn
8     from dotenv import load_dotenv
9
10    load_dotenv()
11
12    # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
13    logging.basicConfig(
14        level=logging.INFO,
15        format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
16        datefmt='%H:%M:%S'
17    )
18    logger = logging.getLogger(__name__)
19    from fastapi import FastAPI, HTTPException, Request
20    from fastapi.middleware.cors import CORSMiddleware
21    from pydantic import BaseModel
22    from typing import List, Dict, Any, Optional
23
24    from fastapi.staticfiles import StaticFiles
25    from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
26    from backend.pipeline.validators.base import registry
27    from backend.infrastructure.database import Database
28    from backend.pipeline.segmenters.base import segmenter_registry
29    from backend.pipeline.stitcher.compiler import VideoCompiler
30    from backend.utils.hashing import compute_video_id # canonical file-identity hashing
31    from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
32    from backend.config import ( # new typed config
33        load_config as load_app_config,
34        AppConfig,
35        StitchingConfig,
36        enforce_startup_checks,
37        StartupCheckError,
38    )
39    from backend.results import Failure # standardized error/result contract
40    from backend.reporting import emit_indexer_report
41    from backend import storage # M2: URI-aware input acquisition (local = identity
passthrough)
42    from backend import billing # M8: per-job cost ledger (USD internal / INR display)
43    from backend.api import auth as edge_auth # M9: Cloudflare Access edge-auth (default-
OFF)
44
45    app = FastAPI(title="Sports Highlight Indexer API")
46
47
48    def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
49        """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-
separated),
50        default ``["*"]``. Read from the ENV (not a cwd ``config.json``) so importing
``backend.main``
51        does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to
its origin."""
52        src = os.environ if env is None else env
53        raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
54        origins = [o.strip() for o in raw.split(",") if o.strip()]
55        return origins or ["*"]
56
57
58    # CORS for the web UI. allow_origins='' with allow_credentials=True is rejected by
browsers per the
59    # fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so
credentials stay
60    # off. The origin list is configurable via the env var above (default ["*"]) so a hosted
deploy can

```

```

61     # lock it to its origin; the middleware is fixed at startup.
62     app.add_middleware(
63         CORSMiddleware,
64         allow_origins=_cors_origins_from_env(),
65         allow_credentials=False,
66         allow_methods=["*"],
67         allow_headers=["*"],
68     )
69
70     # Serves static frontend files directly (now under api/ per approved structure)
71     os.makedirs("backend/api/static", exist_ok=True)
72     app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
73
74     @app.get("/", response_class=HTMLResponse)
75     def serve_index():
76         index_path = "backend/api/static/index.html"
77         if os.path.exists(index_path):
78             with open(index_path, "r", encoding="utf-8") as f:
79                 return f.read()
80         return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
81
82     # Global variables initialized at startup
83     db_instance: Optional[Database] = None
84     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
85
86     # Async job worker ? drain/wake loop (DEFERRED_HARDENING_PLAN #16, EXECUTION_POLICY.md
P3)
87     # now lives in backend.api.job_worker. Re-exported here so the names stay addressable on
88     # `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`, `_get_executor`,
89     # `_run_claimed_job`, `_MAX_DRAIN_WAKE_SEC`, ...). The worker resolves these seams
THROUGH
90     # this module object at call-time, so `monkeypatch.setattr(backend.main, ...)` still
91     # intercepts the inter-function calls (a parked job arms the PATCHED `_arm_wake_timer`,
92     # so no real daemon timer leaks in tests). Behavior is unchanged by the extraction.
93     from backend.api.job_worker import ( # noqa: F401 (re-exports: names stay addressable
on backend.main)
94         JobWaiting,
95         _arm_wake_timer,
96         _drain_due_jobs,
97         _get_executor,
98         _job_to_request,
99         _MAX_DRAIN_WAKE_SEC,
100         _maybe_record_job_cost,

```

11. user (2026-06-21T14:59:35.693Z)

```

1     """Path-safety helpers for the API boundary (untrusted request fields)."""
2     import os
3     from typing import Any, Optional
4
5
6     def output_base(config: Any) -> str:
7         """The configured base dir for a segmenter's LOCAL scratch/staging artifacts
8         (COMPUTE_DECOUPLED_SERVING M3). Returns ``output.output_dir`` (default ``"output"``)
so the
9         chunk-staging / AI-handoff / temp dirs follow ``--output-dir`` /
``output.output_dir`` instead of
10         a hardcoded ``./output``. The DEFAULT is ``"output"`` ? byte-identical to the prior
literal.
11
12         This is the LOCAL output dir, deliberately NOT the storage ``workspace_root`` (which
may be a
13         ``gs://`` URI you can't ``mkdirs``). Accepts the typed AppConfig or the legacy
dict-shim

```

```

14         (``.get("output", {}).get("output_dir")``); any miss or empty value falls back to
15         ``"output"``. """
16         try:
17             out = config.get("output", {}).get("output_dir")
18         except Exception:
19             out = None
20         return out or "output"
21
22     def safe_output_filename(name: str) -> str:
23         """Validate an output filename is a BARE name (no dir part, no traversal, not
24         absolute).
25         `os.path.join(output_dir, name)` silently discards `output_dir` when `name` is
26         absolute,
27         and `..` segments escape it ? so an unchecked `output_filename` is an arbitrary-
28         write
29         primitive. Returns the name unchanged when safe; raises ValueError otherwise.
30         """
31         if not name or not name.strip():
32             raise ValueError("output_filename must not be empty")
33         if "\x00" in name:
34             raise ValueError("output_filename must not contain a null byte")
35         if "/" in name or "\\" in name:
36             raise ValueError(f"output_filename must not contain a path separator: {name!r}")
37         if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
38             raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
39         return name
40
41     def resolve_under_root(path: str, root: str) -> str:
42         """Return the real path, asserting it is contained under `root` (raises ValueError
43         if not). """
44         rp = os.path.realpath(path)
45         rr = os.path.realpath(root)
46         try:
47             contained = os.path.commonpath([rp, rr]) == rr
48         except ValueError: # different drives on Windows
49             contained = False
50         if not contained:
51             raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
52         return rp
53
54     def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
55         """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.
56
57         With no allowed_root (default, single-user local mode) only the null-byte check
58         applies,
59         so the local 'point it at any of my videos' workflow is preserved. Setting an
60         allowed_root (a future hosted/tenant deployment) enforces containment.
61         """
62         if "\x00" in path:
63             raise ValueError("video_path must not contain a null byte")
64         if allowed_root:
65             resolve_under_root(path, allowed_root)

```

12. assistant (2026-06-21T14:59:37.971Z)

Now let me search for how the other routes (/api/compile, /api/highlights) handle the allowed_video_root:

13. user (2026-06-21T14:59:39.100Z)

Found 1 file
backend\main.py

14. user (2026-06-21T14:59:39.924Z)

```
350         per-job rows. A video is 1:1 with an infer job, but a re-ingest can produce
351         several.""
352         agg = db_instance.get_video_cost(video_id)
353         fx = global_config.billing.fx_inr_per_usd
354         agg["total_cost_inr"] = billing.to_inr(agg["total_cost_usd"], fx)
355         agg["fx_inr_per_usd"] = fx
356         return agg
357
358     @app.get("/api/jobs/{job_id}")
359     def get_job(job_id: str):
360         """Job state: {status, progress, result, reports}. `status` = execution state
361         (queued|running|done|failed); the pipeline verdict is result["status"]."""
362         job = db_instance.get_job(job_id)
363         if not job:
364             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
365         result = job.get("result")
366         return {
367             "job_id": job["id"],
368             "status": job["status"],
369             "progress": job.get("progress"),
370             "video_id": job.get("video_id"),
371             "error": job.get("error"),
372             "result": result,
373             "reports": (result or {}).get("reports"),
374             "created_at": job.get("created_at"),
375             "started_at": job.get("started_at"),
376             "finished_at": job.get("finished_at"),
377         }
378
379     # --- Chunked upload (B2, PR-2) -----
380     # The chunked-upload island (in-process registry + helpers + the 4 endpoints) now lives
381     # in
382     # backend.api.uploads as a self-contained APIRouter. It resolves the live startup config
383     # via
384     # backend.main.global_config, so behavior ? route paths, status codes, body-size limits,
385     # sequential-part logic, and abort cleanup ? is unchanged.
386     from backend.api import uploads as uploads_api
387
388     app.include_router(uploads_api.router)
389
390     # --- Highlight download (B3, PR-2) -----
391
392     @app.get("/api/highlights/{video_id}/{filename}")
393     def download_highlight(video_id: str, filename: str):
394         """Stream a compiled highlight reel ? `filename` is the bare name given to
395         /api/compile
396         (which only writes into output_dir; the browser previously got back an unreachable
397         server-local path)."""
398         try:
399             name = safe_output_filename(filename)
400         except ValueError as e:
401             raise HTTPException(status_code=400, detail=str(e)) from e
402         if not db_instance.get_video(video_id):
403             raise HTTPException(status_code=404, detail="Indexed video record not found.")
404         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
405         or "output"
406         path = os.path.join(output_dir, name)
407         try:
```

```

405         path = resolve_under_root(path, output_dir)
406     except ValueError as e:
407         raise HTTPException(status_code=400, detail=str(e)) from e
408     if not os.path.isfile(path):
409         raise HTTPException(status_code=404,
410                             detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
411     media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
412     return FileResponse(path, media_type=media, filename=name)
413
414
415 @app.post("/api/query")
416 def query_play_segments(req: QueryRequest):
417     filters = {
418         "server": req.server,
419         "receiver": req.receiver,
420         "duration_min": req.duration_min,
421         "event_type": req.event_type
422     }
423     segments = db_instance.query_play_segments(req.video_id, filters)
424     return {"play_segments": segments}
425
426 @app.post("/api/compile")
427 def compile_highlights(req: CompileRequest):
428     video = db_instance.get_video(req.video_id)
429     if not video:

```

15. user (2026-06-21T14:59:42.826Z)

```

426 @app.post("/api/compile")
427 def compile_highlights(req: CompileRequest):
428     video = db_instance.get_video(req.video_id)
429     if not video:
430         raise HTTPException(status_code=404, detail="Indexed video record not found.")
431
432     # Retrieve matching play segments from db
433     all_segments = db_instance.query_play_segments(req.video_id, {})
434     matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
435
436     if not matched_segments:
437         raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
438
439     # Reject path traversal / absolute output names (os.path.join would otherwise let an
440     # absolute or '..'-laden output_filename write anywhere on disk).
441     try:
442         out_name = safe_output_filename(req.output_filename)
443     except ValueError as e:
444         raise HTTPException(status_code=400, detail=str(e)) from e
445
446     # The configured shot-count floor (stitching.min_shot_count) applies on the API
447     # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
shot_count
448     # metadata are exempt: the floor filters known counts only, so explicitly
449     # requested manual/legacy segments can't silently vanish under the default floor.
450     stitching = getattr(global_config, "stitching", None) or StitchingConfig()
451     kept = []
452     for s in matched_segments:
453         meta = s.get("metadata") or {}
454         sc = meta.get("shot_count") if isinstance(meta, dict) else None
455         try:
456             if sc is not None and int(sc) < stitching.min_shot_count:
457                 continue
458         except (TypeError, ValueError):
459             pass # unparseable count -> treat as unknown, keep

```

```

460         kept.append(s)
461     if len(kept) < len(matched_segments):
462         logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped "
463                     f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")
464     if not kept:
465         raise HTTPException(status_code=400,
466                             detail=f"All {len(matched_segments)} matching segments fall
below "
467                                 f"stitching.min_shot_count={stitching.min_shot_count}.")
468
469     # Extract start/end time pairs
470     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
471
472     # Run compiler with the configured stitching paddings + optional branding watermark
473     output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
474     compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
475                             end_padding=stitching.end_padding,
watermark=stitching.watermark)
476     try:
477         # The stored `filepath` is the SOURCE ref ? a bucket/Drive URI (gs://, s3://,
gdrive://) for a
478         # video ingested via /api/process from a URI, or a plain local path. Resolve it
to a LOCAL file
479         # the stitcher can read: identity for a local path (byte-identical to before),
fetch-to-scratch
480         # for a bucket URI (the same seam the process path uses, _execute_processing
main.py:159).
481         # Without this, compiling a bucket-ingested video 500s ? ffmpeg can't open a
gs:// path.
482         local_src = storage.acquire_local_input(video["filepath"],
getattr(global_config, "storage", None))
483         out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
484         # `filepath` is server-local (not reachable by the browser); also return the
ready-to-use
485         # download URL so the SPA needn't hand-build it (P6 / PER_RALLY_SELECTION.md
§4.3).

```

16. user (2026-06-21T14:59:43.470Z)

Found 4 files
backend\config\models.py
artifacts\so.py
tests\test_upload_download.py
backend\api\uploads.py

17. user (2026-06-21T14:59:46.007Z)

```

1     """Chunked upload (B2, PR-2) ? extracted from backend.main into its own router.
2
3     Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
4     proxies cap request bodies at ~100 MB ? HOSTING_PLAN §2), are assembled server-side
5     under security.upload_dir, then submitted to /api/process by the returned path.
6     Upload state is in-process by design (this is the single-box alpha): a server restart
7     aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
8     arrives with PR-3 identity scoping.
9
10    The startup config lives on ``backend.main.global_config`` (set ONLY in ``main()``).
11    ``_upload_cfg`` resolves it through the ``backend.main`` module object so it always
12    reads the live value the server set at startup (and so tests that monkeypatch
13    ``backend.main.global_config`` flow through unchanged).
14    """

```

```

15     import os
16     import threading
17     import uuid
18     from typing import Any, Dict, Optional
19
20     from fastapi import APIRouter, HTTPException, Request
21     from pydantic import BaseModel
22
23     from backend.utils.hashing import compute_video_id # canonical file-identity hashing
24     from backend.utils.paths import safe_output_filename
25
26     router = APIRouter()
27
28     _uploads: Dict[str, Dict[str, Any]] = {}
29     _uploads_lock = threading.Lock()
30
31
32     class UploadInitRequest(BaseModel):
33         filename: str
34         total_size_bytes: Optional[int] = None
35
36
37     class UploadCompleteRequest(BaseModel):
38         total_parts: int
39
40
41     def _upload_cfg():
42         # Resolved lazily (import inside the function) to read the LIVE startup config that
43         # main() sets on the module global, and to avoid a circular import at module load
44         # (main.py imports this router; this module must not import main at import time).
45         import backend.main as _main
46         sec = getattr(_main.global_config, "security", None)
47         upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
48         max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
49         part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
50         exts = [e.lower().lstrip(".") for e in (getattr(sec, "allowed_upload_extensions",
51 None) or ["mp4", "mov"])]
52         return upload_dir, max_bytes, part_bytes, exts
53
54     def _abort_upload(upload_id: str) -> None:
55         with _uploads_lock:
56             st = _uploads.pop(upload_id, None)
57             if st:
58                 try:
59                     os.remove(st["tmp_path"])
60                 except OSError:
61                     pass
62
63
64     @router.post("/api/upload/init")
65     def upload_init(req: UploadInitRequest):
66         """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
67         upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
68         try:
69             name = safe_output_filename(req.filename)
70         except ValueError as e:
71             raise HTTPException(status_code=400, detail=str(e)) from e
72         ext = os.path.splitext(name)[1].lower().lstrip(".")
73         if ext not in exts:
74             raise HTTPException(status_code=400,
75 detail=f"Extension '{ext}' not allowed. Allowed:
76 {sorted(exts)}")
77         if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
78             raise HTTPException(status_code=413,

```

```

78             detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB
upload limit.")
79         os.makedirs(upload_dir, exist_ok=True)
80         upload_id = uuid.uuid4().hex
81         tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
82         open(tmp_path, "wb").close()
83         with _uploads_lock:
84             _uploads[upload_id] = {"filename": name, "tmp_path": tmp_path, "parts": 0,
"bytes": 0}
85         return {"upload_id": upload_id, "max_part_mb": part_bytes // (1024 * 1024),
86                 "next_part": f"/api/upload/{upload_id}/part/0"}
87
88
89     @router.put("/api/upload/{upload_id}/part/{index}")
90     async def upload_part(upload_id: str, index: int, request: Request):
91         """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
92         _, max_bytes, part_bytes, _ = _upload_cfg()
93         with _uploads_lock:
94             st = _uploads.get(upload_id)
95             if st is None:
96                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
97             if index != st["parts"]:
98                 raise HTTPException(status_code=409,
99                                     detail=f"Out-of-order part: expected {st['parts']}, got
{index} (parts are sequential)")
100
101             received = 0
102             overflow = None
103             with open(st["tmp_path"], "ab") as fh:
104                 async for chunk in request.stream():
105                     received += len(chunk)
106                     if received > part_bytes:
107                         overflow = f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
limit."
108                         break
109                     if st["bytes"] + received > max_bytes:
110                         overflow = f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
limit."
111                         break
112                     fh.write(chunk)
113             if overflow:
114                 _abort_upload(upload_id)
115                 raise HTTPException(status_code=413, detail=overflow + " Upload aborted ? re-
init to retry.")
116
117             with _uploads_lock:
118                 st["parts"] += 1
119                 st["bytes"] += received
120             return {"upload_id": upload_id, "received_parts": st["parts"], "received_bytes":
st["bytes"]}
121
122
123     @router.post("/api/upload/{upload_id}/complete")
124     def upload_complete(upload_id: str, req: UploadCompleteRequest):
125         """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
126         upload_dir, _, _, _ = _upload_cfg()
127         with _uploads_lock:
128             st = _uploads.get(upload_id)
129             if st is None:
130                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
131             if st["parts"] == 0 or st["parts"] != req.total_parts:
132                 raise HTTPException(status_code=409,
133                                     detail=f"Expected {req.total_parts} part(s), received
{st['parts']}.")

```

```

134     final_path = os.path.join(upload_dir, st["filename"])
135     if os.path.exists(final_path): # never overwrite an earlier upload with the same
name
136         stem, ext = os.path.splitext(st["filename"])
137         final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
138     os.replace(st["tmp_path"], final_path)
139     with _uploads_lock:
140         _uploads.pop(upload_id, None)
141     video_id = compute_video_id(final_path)
142     return {"path": os.path.abspath(final_path), "video_id": video_id,
143           "size_bytes": os.path.getsize(final_path),
144           "next": "POST /api/process with this path"}
145
146
147     @router.delete("/api/upload/{upload_id}")
148     def upload_abort(upload_id: str):
149         """Abort an in-flight upload and delete its partial file."""
150         with _uploads_lock:

```

18. assistant (2026-06-21T14:59:48.189Z)

Now let me check if the upload_dir path is validated when it's returned to /api/process:

19. user (2026-06-21T14:59:48.962Z)

Found 1 file
artifacts\so.py

20. user (2026-06-21T14:59:49.883Z)

```

1     """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3     python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4         [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5     python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7     The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
``local://``
8     is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
local runs
9     are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
10    ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
key) ?
11    the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
12
13    This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
CLI uses
14    (validator registry ? segmenter ? report), never a forked pipeline.
15    """
16    from __future__ import annotations
17
18    import argparse
19    import csv
20    import json
21    import logging
22    import os
23    import sys
24    import tempfile
25    from typing import Any, List
26
27    from backend import storage
28    from backend.compute import ComputeJobSpec, get_compute_backend
29    from backend.config import (
30        StartupCheckError,

```

```

31         enforce_startup_checks,
32         load_config,
33         probe_cuda_available,
34     )
35     from backend.infrastructure.database import Database
36     from backend.pipeline.segmenters.base import segmenter_registry
37     from backend.pipeline.validators.base import registry as validator_registry
38     from backend.reporting import emit_indexer_report
39     from backend.results import Failure
40     from backend.utils.hashing import compute_video_id
41
42     logger = logging.getLogger("backend.worker")
43
44
45     def _coerce(v: str) -> Any:
46         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
47         str."""
48         low = v.lower()
49         if low in ("true", "false"):
50             return low == "true"
51         for cast in (int, float):
52             try:
53                 return cast(v)
54             except ValueError:
55                 pass
56         return v
57
58     def _apply_overrides(config: Any, sets: List[str]) -> None:
59         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
60         indexing.skip_ai_handoff=true)."""
61         for kv in sets or []:
62             key, sep, val = kv.partition("=")
63             if not sep:
64                 raise ValueError(f"--set expects k=v, got {kv!r}")
65             parts = key.split(".")
66             obj = config
67             for p in parts[:-1]:
68                 obj = getattr(obj, p)
69             setattr(obj, parts[-1], _coerce(val))
70
71     def _write_intervals_csv(segments: List[dict], path: str) -> None:
72         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
73         friendly
74         artifact (same schema as the rally-annotator labels)."""
75         with open(path, "w", newline="") as f:
76             w = csv.writer(f)
77             w.writerow(["start", "end", "shot_count", "ending_reason"])
78             for s in segments:
79                 w.writerow([
80                     f'{float(s.get("start_time", 0.0)):.3f}',
81                     f'{float(s.get("end_time", 0.0)):.3f}',
82                     s.get("shot_count", ""),
83                     s.get("ending_reason", s.get("shot_count_confidence", "")),
84                 ])
85
86     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
87     None:
88         with open(path, "w") as f:
89             json.dump({
90                 "video_id": video_id,
91                 "status": status,
92                 "segment_count": len(segments),

```

```

92         "segments": [{"start": float(s.get("start_time", 0.0)),
93                        "end": float(s.get("end_time", 0.0))} for s in segments],
94     }, f, indent=2)
95
96
97     def _run_startup_checks(config: Any) -> bool:
98         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
99         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
100         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op

```

21. user (2026-06-21T14:59:53.035Z)

```

97     def _run_startup_checks(config: Any) -> bool:
98         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
99         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
100         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
101         (returns True, ZERO work) when no deployment.profile is selected.
102
103         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
104         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
105         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
106         no-op of work, not just of output."""
107         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
108         cuda = probe_cuda_available() if profile else None # torch-free on the default-OFF
path
109         try:
110             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
111             return True
112         except StartupCheckError:
113             return False # already logged at ERROR by enforce_startup_checks
114
115
116     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
117         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the
118         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
119         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
120
121         A remote job that never writes a completion marker (a failed/hung container) makes
the bucket
122         poll exhaust its budget ? ``PolicyTimeout``. Catch it and return a CLEAN rc 4
(remote dispatch
123         timeout/failure, distinct from the local 2/3) rather than letting a raw traceback
escape."""
124         from backend.infrastructure.execution_policy import PolicyTimeout
125
126         spec = ComputeJobSpec(

```

22. user (2026-06-21T14:59:53.815Z)

```

1     """Startup checks ? per-profile coherence (M5) + exposure-safety posture (M9 hardening).
2
3     A deployment profile (``truly-local`` / ``cloud-serving`` / ``cpu-mock``) implies a
coherent

```

```

4      runtime. These checks **fail/warn fast at process startup** ? a loud signal *before* any
5      (expensive) job runs ? when the active profile is inconsistent with the environment:
6
7      * **cloud-serving with a non-cloud storage backend** ? a hard **error** (the one
config bug we
8      can detect with certainty + zero false-positive risk: a cloud profile that can't
reach a
9      bucket can never serve). This is the README §4.3 "loud error before any job".
10     * a **GPU mismatch** (e.g. ``device='cuda'`` on a box where CUDA isn't visible, or
``device='auto'``
11     falling back to CPU) ? a **warning**, never an error: the probe is best-effort
(torch may live
12     only in the detector subprocess env ? see ``presets.probe_cuda_available``), and the
detector's
13     own healthcheck is the real authority. A warning surfaces it early without false-
failing.
14     * **creds that can't be confirmed** (no ``GOOGLE_APPLICATION_CREDENTIALS`` / not on
Cloud Run) ?
15     a **warning**: Application Default Credentials can come from the metadata server /
gcloud, which
16     we can't probe offline, so the storage backend's own init is the authority; we just
flag it.
17
18     **Exposure-safety posture (server only, ``include_exposure=True``). When the engine is
*exposed*
19     behind an edge gate (the owner flips ``security.edge_auth_enabled`` on for the
Cloudflare-Access
20     boundary ? the alpha-shareable "tunnel-to-a-box" topology), the gate is only half the
safety story.
21     A half-configured exposure starts *looking* healthy then leaks / 500s per request, so we
verify the
22     rest of the posture at **boot**: the ``allowed_video_root`` path-jail is set (else an
authenticated
23     tenant could read an arbitrary local file ? **error**), the CF team-domain + AUD are
present (else
24     no token can verify ? **error**, lifting the current first-request failure to boot), and
the
25     ``auth`` extra is importable (else every request 500s ? **warning**). Only the HTTP
**server** passes
26     ``include_exposure=True``; the worker reads ``--video-uri`` (not untrusted tenant
paths), so it is
27     never "exposed" and stays unaffected.
28
29     **DEFAULT-OFF (both dimensions):** with no profile selected (``profile == ""``) the
profile checks are
30     a strict **no-op** ? exactly the pre-M5 behaviour; with ``edge_auth_enabled=False``
(today's default)
31     the exposure checks are a strict **no-op** too. So nothing changes for today's manual-
knob deployments.
32     ""
33
34     from __future__ import annotations
35
36     import logging
37     import os
38     from dataclasses import dataclass
39     from typing import Any, Mapping, Optional
40
41     LEVEL_ERROR = "error"
42     LEVEL_WARN = "warn"
43
44     # Storage backends that count as "a cloud bucket" for cloud-serving coherence.
45     _CLOUD_STORAGE = ("gcs", "s3")
46     # STRONG env signals that Google ADC *might* resolve. Deliberately NOT
GOOGLE_CLOUD_PROJECT /

```

```
47      # CLOUDSDK_CONFIG ? those are often set without usable creds (a plain project id / a
48      # config dir
49      # with no active login), so they would suppress the heads-up falsely. (s3 creds are
50      # intentionally
51      # left to boto3's default chain ? no parallel heads-up, by design.)
52      _GCP_ENV_HINTS = ("GOOGLE_APPLICATION_CREDENTIALS", "K_SERVICE", "GCE_METADATA_HOST")
```

23. assistant (2026-06-21T14:59:55.810Z)

Now let me check what `resolve_tenant` does and verify the static file serving:

24. user (2026-06-21T14:59:56.619Z)

Found 1 file
backend\api\auth.py

25. user (2026-06-21T14:59:57.687Z)

```
100
101
102     def resolve_tenant(headers: Mapping[str, str], config: Any, *,
103                        jwks_fetcher: Optional[JwksFetcher] = None) -> Optional[str]:
104         """Resolve the tenant ``user_id`` for a request, or ``None`` when edge auth is OFF.
105
106         DEFAULT-OFF (``security.edge_auth_enabled=False``) ? ``None`` (the local single-
107         user; unchanged).
108         Enabled ? require + verify the Cf-Access header; raise ``EdgeAuthError`` (? 401/403)
109         on any miss.
110         ``jwks_fetcher`` is injectable for offline tests.
111         """
112         sec = getattr(config, "security", None)
113         if not sec or not getattr(sec, "edge_auth_enabled", False):
114             return None # DEFAULT-OFF ? no tenant identity, byte-identical to today
115
116         team_domain = getattr(sec, "cf_team_domain", "") or ""
117         aud = getattr(sec, "cf_access_aud", "") or ""
118         if not team_domain or not aud:
119             raise EdgeAuthError(
120                 "edge_auth_enabled but security.cf_team_domain / cf_access_aud are not
121                 configured")
122
123         # Headers are case-insensitive over HTTP; Starlette's Headers handles that, but
124         # support a plain
125         # dict too (tests) by trying both casings.
126         token = headers.get(CF_ACCESS_HEADER) or headers.get(CF_ACCESS_HEADER.lower())
127         if not token:
128             raise EdgeAuthError(f"missing {CF_ACCESS_HEADER} header (request did not pass
129             Cloudflare Access)")
130
131         jwks = _get_jwks(team_domain, jwks_fetcher)
132         return verify_cf_access_token(token, team_domain=team_domain, aud=aud, jwks=jwks)
```

26. user (2026-06-21T15:01:06.557Z)

C:\Users\avidu\Projects\badminton-highlight-indexer/backend/api/auth.py

27. assistant (2026-06-21T15:01:08.559Z)

Now let me examine how static files are served and check if there are other untrusted path entry points:

28. user (2026-06-21T15:01:09.364Z)

Found 1 file
backend\main.py

29. user (2026-06-21T15:01:10.114Z)

```
70     # Serves static frontend files directly (now under api/ per approved structure)
71     os.makedirs("backend/api/static", exist_ok=True)
72     app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
73
74     @app.get("/", response_class=HTMLResponse)
75     def serve_index():
76         index_path = "backend/api/static/index.html"
77         if os.path.exists(index_path):
78             with open(index_path, "r", encoding="utf-8") as f:
79                 return f.read()
```

30. assistant (2026-06-21T15:01:12.172Z)

Now let me check the test file to understand what exactly is tested:

31. user (2026-06-21T15:01:13.035Z)

```
238     """A warning-only profile (truly-local + cloud storage) must NOT abort the
server."""
239     import backend.main as bmain
240     assert bmain._enforce_startup_or_exit(_cfg(profile="truly-local", backend="gcs")) is
None
241
242
243     # ----- exposure-safety posture (M9 hardening,
server-only)
244
245     def test_exposure_off_by_default_is_noop():
246         """edge_auth_enabled=False (today's default) ? ZERO exposure checks, even with
include_exposure
247         and an otherwise-unsafe config (no allowed_video_root). The strict default-OFF
guarantee."""
248         cfg = _exposed_cfg(edge_auth=False, video_root=None, team="", aud="")
249         assert run_startup_checks(cfg, include_exposure=True, env={}) == []
250
251
252     def test_exposure_not_run_without_flag(monkeypatch):
253         """The worker path (include_exposure=False, the default) NEVER runs exposure checks
? even with
254         edge auth ON and an unsafe config. Only the HTTP server opts in."""
255         monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
256         cfg = _exposed_cfg(edge_auth=True, video_root=None, team="", aud="")
257         assert run_startup_checks(cfg, env={}) == [] # include_exposure defaults False
258
259
260     def test_exposed_without_video_root_is_fatal(monkeypatch):
261         monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
262         cfg = _exposed_cfg(video_root=None) # team+aud set, jail missing
263         checks = run_startup_checks(cfg, include_exposure=True, env={})
264         assert _codes(checks) == {"EXPOSED_NO_VIDEO_ROOT"}
265         assert checks[0].level == LEVEL_ERROR
266
267
268     @pytest.mark.parametrize("video_root", [None, "", " "])
269     def test_exposed_blank_video_root_is_fatal(monkeypatch, video_root):
270         """None, empty, and whitespace-only roots are all treated as 'unset' ? fatal."""
271         monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
272         cfg = _exposed_cfg(video_root=video_root)
273         assert "EXPOSED_NO_VIDEO_ROOT" in _codes(run_startup_checks(cfg,
```

```

include_exposure=True, env={}))
274
275
276 @pytest.mark.parametrize("team,aud,expect_missing", [
277     ("", "abc", "cf_team_domain"),
278     ("https://t.cloudflareaccess.com", "", "cf_access_aud"),
279     ("", "", "cf_team_domain + cf_access_aud"),
280 ])
281 def test_exposed_incomplete_cf_config_is_fatal(monkeypatch, team, aud, expect_missing):
282     monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
283     cfg = _exposed_cfg(team=team, aud=aud) # jail set, CF config partial
284     checks = run_startup_checks(cfg, include_exposure=True, env={})
285     assert _codes(checks) == {"EXPOSED_EDGE_AUTH_INCOMPLETE"}
286     assert checks[0].level == LEVEL_ERROR
287     assert expect_missing in checks[0].message # names exactly what's missing
288
289
290 def test_exposed_missing_auth_extra_warns(monkeypatch):
291     """PyJWT not importable ? a WARN (never fatal ? the auth module's ImportError is the
authority)."""
292     monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: False)
293     cfg = _exposed_cfg() # fully configured otherwise
294     checks = run_startup_checks(cfg, include_exposure=True, env={})
295     assert _codes(checks) == {"EXPOSED_AUTH_EXTRA_MISSING"}
296     assert checks[0].level == LEVEL_WARN
297
298
299 def test_exposed_fully_configured_is_clean(monkeypatch):
300     monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
301     cfg = _exposed_cfg() # edge auth on, jail set, team+aud set, auth extra present
302     assert run_startup_checks(cfg, include_exposure=True, env={}) == []
303
304
305 def test_exposed_both_errors_together(monkeypatch):
306     """An unsafe exposure trips BOTH fatal checks at once (jail missing + CF config
missing)."""
307     monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
308     cfg = _exposed_cfg(video_root=None, team="", aud="")
309     codes = _codes(run_startup_checks(cfg, include_exposure=True, env={}))
310     assert codes == {"EXPOSED_NO_VIDEO_ROOT", "EXPOSED_EDGE_AUTH_INCOMPLETE"}
311
312
313 def test_exposure_composes_with_profile_checks(monkeypatch):
314     """Exposure posture is INDEPENDENT of deployment.profile ? an exposed cloud-serving
config with
315     local storage trips both the exposure jail error AND the profile STORAGE_INCOHERENT
error."""
316     monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
317     cfg = _exposed_cfg(video_root=None, profile="cloud-serving", backend="local")
318     codes = _codes(run_startup_checks(cfg, cuda_available=True, include_exposure=True,
319                                     env={"K_SERVICE": "x"}))
320     assert codes == {"EXPOSED_NO_VIDEO_ROOT", "STORAGE_INCOHERENT"}
321
322
323 def test_server_enforce_exits_on_unsafe_exposure(monkeypatch):
324     """The server's fail-fast wiring aborts (exit 1) when edge auth is on but the engine
is exposed
325     without the allowed_video_root jail ? BEFORE serving a single request."""
326     monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
327     import backend.main as bmain
328     with pytest.raises(SystemExit) as ei:
329         bmain._enforce_startup_or_exit(_exposed_cfg(video_root=None))
330     assert ei.value.code == 1
331
332

```

```

333     def test_server_enforce_allows_safe_exposure(monkeypatch):
334         """A fully-configured exposure starts cleanly (no exit)."""
335         monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
336         import backend.main as bmain
337         assert bmain._enforce_startup_or_exit(_exposed_cfg()) is None
338
339
340     def test_worker_path_ignores_unsafe_exposure(monkeypatch, tmp_path):
341         """The worker (cmd path) does NOT enforce exposure posture ? an edge-auth-on config
that would
342         abort the server runs fine on the worker (it reads --video-uri, not untrusted tenant
paths)."""
343         monkeypatch.setattr(startup_mod, "_auth_extra_available", lambda: True)
344         from backend.worker import __main__ as wmain
345         cfg_file = tmp_path / "c.json"
346         cfg_file.write_text('{"security": {"edge_auth_enabled": true}}') # exposed-ish, no
jail/team/aud
347         assert wmain._run_startup_checks(wmain.load_config(str(cfg_file))) is True
348

```

32. assistant (2026-06-21T15:01:15.030Z)

Now let me search for other endpoints that might accept untrusted paths or query parameters:

33. user (2026-06-21T15:01:16.143Z)

```

74:@app.get("/", response_class=HTMLResponse)
144:@app.get("/api/config")
148:@app.get("/api/health")
281:@app.post("/api/process", status_code=202)
332:@app.get("/api/jobs/{job_id}/cost")
347:@app.get("/api/videos/{video_id}/cost")
358:@app.get("/api/jobs/{job_id}")
391:@app.get("/api/highlights/{video_id}/{filename}")
415:@app.post("/api/query")
426:@app.post("/api/compile")

```

34. assistant (2026-06-21T15:01:18.437Z)

Let me check if there are other routes or if any untrusted input leaks through. Let me also verify the comment about upload_dir needing to be within allowed_video_root:

35. user (2026-06-21T15:01:20.024Z)

```

285- # --- Chunked upload (B2, PR-2) ---
286- # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
287- # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
288- # contain upload_dir or /api/process will reject the uploaded paths.
289- upload_dir: str = "output/uploads"
290- # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
291- # free-tier edge proxies cap request bodies there (HOSTING_PLAN $2).
292- max_upload_mb: int = 4096

```

36. assistant (2026-06-21T15:01:22.262Z)

Perfect. Now let me create a comprehensive analysis. Let me check one more thing - whether the check actually validates that upload_dir is contained within allowed_video_root:

37. user (2026-06-21T15:01:23.142Z)

No files found

38. assistant (2026-06-21T15:01:25.277Z)

Now I have all the information I need. Let me compile my findings:

39. user (2026-06-21T15:01:56.345Z)

Structured output provided successfully

40. assistant (2026-06-21T15:02:09.159Z)

Summary

I've completed a **thorough adversarial security review** of the exposure-safety-preflight change. Here are the key findings:

Top Finding: Coverage Gap on upload_dir

The boot check **only** validates `allowed_video_root` for `/api/process` but does NOT verify that `security.upload_dir` is contained within the jail. The config docs (line 287-288) state this is required ("set `allowed_video_root` to contain `upload_dir`"), but there's **no enforced boot check**, making this an operational assumption. When a chunked upload completes, its path is later re-validated when passed to `/api/process`, so there's a runtime guard?but the misconfiguration won't be caught until the first upload attempt.

Recommendation: Add a boot check in `_exposure_checks()` that validates `realpath(upload_dir)` is contained under `realpath(allowed_video_root)` when exposed mode is on.

Design Assumption (Not a Bug)

The exposure posture gate is **only** triggered by `edge_auth_enabled=True`. An owner could theoretically deploy with CF-Access as a network-layer gate but leave `edge_auth_enabled=False`, and these checks would not fire. However, the code and documentation correctly identify this as the "owner flips `edge_auth_enabled` on" signal?this is a deployment topology assumption, not a false negative in the check logic.

Message Precision Opportunity

The `EXPOSED_NO_VIDEO_ROOT` error message says a tenant could read "an arbitrary local file" but doesn't clarify this only applies to `/api/process` (upload assembly and `/api/highlights` downloads are in separate containment zones). Consider tightening the message for operator clarity.

What's Solid

- ? Default-OFF is enforced (both `edge_auth_enabled=False` and `include_exposure=False` produce true no-ops)
- ? Path containment logic (`realpath` + `commonpath` check) correctly defeats symlinks, `..`, and Windows cross-drive escapes
- ? Worker path correctly ignores exposure posture (only the HTTP server enforces it)
- ? Comprehensive test coverage includes both-fatal-errors-together and profile-check composition
- ? The three fatal checks (no jail, missing CF config, missing PyJWT) are all real risks that deserve boot-time failure

The change is **ship-worthy** with the understanding that `upload_dir` containment remains an operational requirement (document or add a boot check as per the major finding).